

Smart Hospital

Queue & Record System

Tugas Besar Algoritma dan Struktur Data

Muhammad Noval • Raufal Ghailan • Fariz Dzaki
Adika Belva • Muhammad Malik Habibi

Agenda Presentasi

01

Latar Belakang & Tujuan

Mengapa proyek ini dibangun

02

Arsitektur Sistem

Struktur folder dan desain modular

03

Keputusan Desain

Mengapa memilih struktur data ini

04

Demo CLI

Contoh perintah dan output program

05

Pembahasan Efisiensi

Analisis Big-O setiap operasi

06

Hasil Eksperimen

Tabel runtime N=50, 200, 500

07

Kesimpulan

Rangkuman dan pencapaian proyek

Latar Belakang



Antrian Tidak Teratur

Pasien KRITIS bisa menunggu lebih lama dari pasien NORMAL karena tidak ada sistem prioritas



Pencatatan Manual

Tindakan dokter sulit di-undo jika terjadi kesalahan pencatatan secara manual

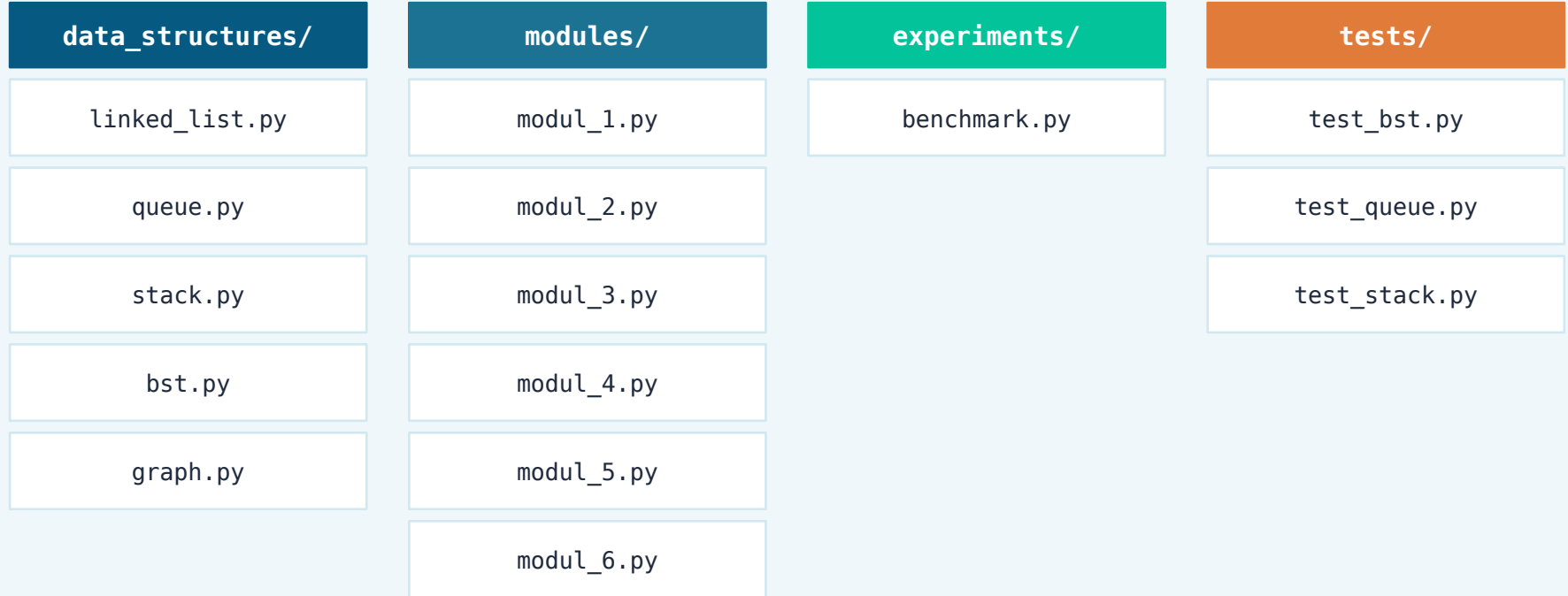


Rekam Medis Lambat

Pencarian rekam medis dari ratusan ribu data memerlukan struktur data yang efisien

Solusi: Sistem CLI berbasis Python dengan Priority Queue, Stack, BST, Linked List, dan Graph yang terintegrasi

Arsitektur Sistem



Setiap modul mengimpor dari data_structures/ → tidak ada duplikasi kode

Keputusan Desain

Linked List	Fondasi semua struktur data Dipisah ke linked_list.py agar bisa diimpor semua modul tanpa duplikasi
Priority Queue	Antrian pasien prioritas KRITIS selalu di depan dengan FIFO sesama KRITIS — enqueue $O(n)$, dequeue $O(1)$
Stack	Log tindakan dokter LIFO cocok untuk fitur undo — push/pop $O(1)$, tiap dokter punya stack terisolasi
BST	Rekam medis cepat Search $O(\log n)$ jauh lebih cepat dari sequential search $O(n)$ untuk data besar
Graph	Koneksi antar poli Adjacency list untuk merepresentasikan rujukan antar poli secara dinamis

Demo — CLI Interaktif

terminal

```
>> DAFTAR Budi umum NORMAL
```

```
Budi masuk antrian umum sebagai NORMAL | big-o: O(n)
```

```
>> DAFTAR Andi umum KRITIS
```

```
Andi masuk antrian umum sebagai KRITIS | big-o: O(n)
```

```
>> PANGGIL umum
```

```
Andi dipanggil dari umum | big-o: O(1)
```

```
>> CARI_RM 101
```

```
ketemu -> Budi | umur: 45 | hipertensi
```

```
big-o: rata-rata O(log n), worst case O(n)
```

```
>> KELUAR
```

```
sistem ditutup
```

Daftar Perintah

DAFTAR	$O(n)$
PANGGIL	$O(1)$
UNDO_DOKTER	$O(1)$
CARI_RM	$O(\log n)$
TAMBAH_RM	$O(\log n)$
LAPORAN_HARI	$O(n)$
KELUAR	$O(1)$

Pembahasan Efisiensi — Big-O

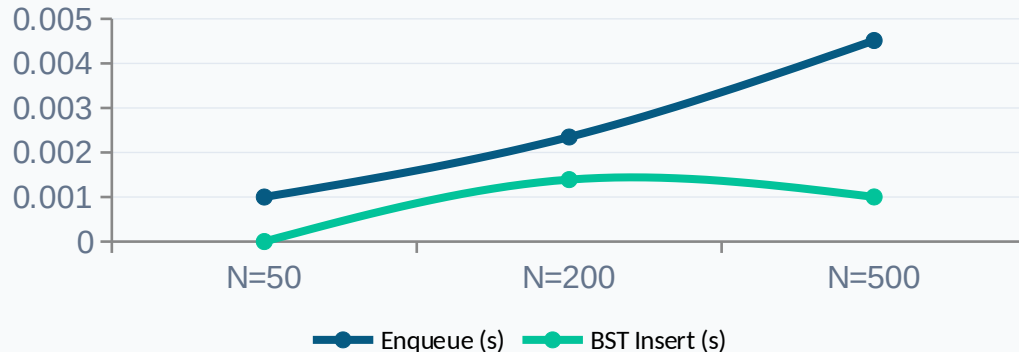
Struktur Data	Operasi	Waktu (avg)	Waktu (worst)	Ruang
Priority Queue	enqueue	$O(n)$	$O(n)$	$O(n)$
Priority Queue	dequeue	$O(1)$	$O(1)$	$O(n)$
Stack	push / pop	$O(1)$	$O(1)$	$O(n)$
BST	insert / search	$O(\log n)$	$O(n)$	$O(n)$
Linked List	tambah / hapus	$O(n)$	$O(n)$	$O(n)$
Graph	tambah_node/edge	$O(1)$	$O(1)$	$O(V+E)$
Insertion Sort	sort	$O(n^2)$	$O(n^2)$	$O(n)$
Selection Sort	sort	$O(n^2)$	$O(n^2)$	$O(n)$

BST worst-case $O(n)$ terjadi bila data dimasukkan berurutan (tree menjadi linier). Solusi: masukkan data secara acak.

Hasil Eksperimen Runtime (seed=42)

N (Pasien)	Enqueue (s)	Dequeue (s)	BST Insert (s)	BST Search (s)
50	0.000999	0.002524	0.000000	0.000000
200	0.002347	0.000000	0.001390	0.000000
500	0.004513	0.000000	0.001001	0.000000

Tren Runtime (detik)



Enqueue naik linear: 0.001s → 0.005s sesuai $O(n)$



Dequeue sangat cepat dan stabil, konfirmasi $O(1)$



BST Search sangat cepat di semua N, konfirmasi $O(\log n)$



0.000000 = waktu < presisi microsecond prosesor modern

Validasi Fungsional

Priority Queue

SESUAI

KRITIS dipanggil duluan, FIFO sesama KRITIS terjaga
poli umum: Andi(KRITIS) → Rudi(KRITIS) → Budi(NORMAL)

BST Rekam Medis

SESUAI

Insert, search, delete, inorder bekerja benar
Inorder menghasilkan urutan ascending nomor RM

Stack Dokter

SESUAI

Undo berhasil menghapus tindakan terakhir
Log tersisa tetapurut dan benar

Sorting Laporan

SESUAI

Insertion Sort: waktu tunggu descending ✓
Selection Sort: no antrian ascending ✓

Penambahan: Linked List & Graph

linked_list.py — Fondasi Semua Modul

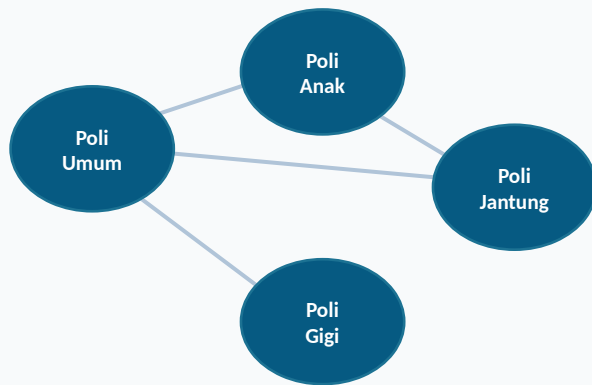
```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
    def tambah(self, data): ...
    def hapus(self, data): ...
```

Semua modul import dari sini:

```
from src.data_structures.linked_list import Node
```

graph.py — Koneksi Antar Poli



Adjacency list — referral antar poli
tambah_node(), tambah_edge(), tampil()

```
from src.data_structures.graph import Graph
```

Kesimpulan

- 1 Priority Queue Multi-Poli berhasil: KRITIS selalu terdepan, enqueue $O(n)$, dequeue $O(1)$
- 2 Stack Dokter berhasil: fitur undo dengan push/pop $O(1)$, tiap dokter terisolasi
- 3 BST Rekam Medis berhasil: search rata-rata $O(\log n)$, terkonfirmasi via eksperimen
- 4 Linked List dipisah sebagai modul fondasi — eliminasi duplikasi kode
- 5 Graph ditambahkan untuk koneksi antar poli menggunakan adjacency list
- 6 CLI dengan 7 perintah + Big-O info setiap eksekusi
- 7 Runtime terkonfirmasi di $N=50, 200, 500$ sesuai teori kompleksitas

Terima Kasih

Pertanyaan & Diskusi

Smart Hospital Queue & Record System

Teknik Elektro — Universitas Negeri Yogyakarta — 2026